

Atribuição, funil e ROI na /trafego — Plano de Implementação

Para executores: use `superpowers:subagent-driven-development` (recomendado) ou `superpowers:executing-plans`. Os passos usam `- [ ]` para acompanhamento.

Goal: Ligar cada paciente ao anúncio que o trouxe, e com esse elo entregar os três blocos que ficaram de fora da Fase 1 da `/trafego`: funil de conversão, ROI por campanha e atribuição por origem.

Architecture: Hoje nada no banco liga um paciente a um anúncio. A WhatsApp Cloud API entrega esse elo de graça: quando alguém chega por um anúncio Click-to-WhatsApp, a **primeira** mensagem vem com um objeto `referral` contendo o ID do anúncio. O webhook já recebe esse payload e o descarta. A espinha do plano é capturar esse dado, traduzir anúncio → campanha, e então derivar as três telas de uma única cadeia:

```
anúncio (referral.source_id)
  ↳ lead_attributions (1 linha por conversa atribuída)
    ↳ conversations.patient_phone
      ↳ appointments (agendado → confirmado → realizado)
        ↳ revenue_entries (amount, payment_status)
```

Funil, ROI e atribuição são três leituras da mesma cadeia. Por isso as Tarefas 1–4 são pré-requisito duro e as Tarefas 6–8 são independentes entre si.

Tech Stack: Next.js 16.3.1 (App Router), TypeScript, Supabase/Postgres, Vitest, Tailwind. Meta Graph API v23.0.

Spec: este plano é autocontido. O design de referência é `Trafego.dc.html` no projeto claude.ai `89bda508-12bf-4624-97dd-8f7e298f35e3`; a Fase 1 (período, KPIs, gráfico, filtros, tabela) já está implementada.

Global Constraints

- Não commitar segredos de `.env.local`.
 - Preferir padrões existentes do projeto a novas abstrações.
 - Todo comportamento novo entra por TDD: teste falhando primeiro, verificado falhando.
 - Toda tabela nova recebe RLS no padrão do projeto (`public.my_account_ids()`).
 - Migrations vão em `supabase/migration/*.sql` e são aplicadas à mão pelo owner.
 - Commits em português, no estilo do histórico (`feat(trafego): ...`).
 - **Não abrir PR.** O usuário abre o PR; o executor só commita.
-

Decisões já tomadas (não reabrir durante a execução)

Estas são as escolhas que normalmente travam uma implementação no meio. Ficam decididas aqui para o executor não precisar adivinhar:

Questão	Decisão	Porquê
Janela de atribuição	90 dias entre o lead e a consulta	Acima disso o crédito ao anúncio vira ficção
Lead que clica em 2 anúncios	Last touch por conversa; guardamos todas as linhas	O último anúncio é o que trouxe a conversa atual
Data de ancoragem do funil	Data do lead , não da consulta	Senão o funil mistura coortes e a taxa não significa nada
ROI usa qual receita	<code>payment_status = 'paid'</code>	ROI é dinheiro no bolso; "previsto" vira uma segunda linha, nunca o número principal
Chave de identidade	<code>patient_phone</code> normalizado E.164	É a única chave que existe nas duas pontas
"Atribuição por origem"	Renomear para " Atribuição por campanha "	Não existe UTM real aqui — ver Tarefa 8

Estrutura de arquivos

Criar

- `supabase/migration_lead_attribution.sql` — tabelas `lead_attributions` e `meta_ad_map`
- `lib/whatsapp/referral.ts` — extrai e normaliza o `referral` do payload
- `lib/trafego/funnel.ts` — agregação do funil
- `lib/trafego/roi.ts` — receita atribuída e ROI
- `lib/trafego/attribution.ts` — leitura da cadeia (query + montagem)
- `components/trafego/FunnelCard.tsx`
- `components/trafego/AttributionList.tsx`
- `scripts/backfill-referrals.sql` — recupera histórico de `webhook_logs`
- Testes espelhados em `tests/whatsapp/` e `tests/trafego/`

Modificar

- `app/api/whatsapp/webhook/route.ts:84` — hoje lê `value.messages[0]` e ignora `referral`
- `lib/whatsapp/process.ts` (ou onde vive `processIncomingMessage`) — repassar a atribuição
- `lib/meta/ads-sync.ts` — segunda chamada em `level: 'ad'` para o mapa anúncio→campanha
- `components/trafego/CampaignsClient.tsx` — 4º KPI, funil, lista de atribuição
- `app/(dashboard)/trafego/page.tsx` — carregar a cadeia de atribuição

Tarefa 0 — Verificar que existe dado antes de construir qualquer coisa

Esta tarefa pode cancelar as outras nove. Faça primeiro.

Todo o plano depende de os anúncios serem **Click-to-WhatsApp**. Um anúncio que manda para o site ou para o Direct do Instagram não gera `referral`, e nesse caso nada aqui preenche — a correção seria na estratégia de anúncio, não no código.

`webhook_logs.payload` guarda o body inteiro do webhook e não há rotina de limpeza em `supabase/cron.sql`. Então o histórico está lá.

☐ Passo 1: Rodar a consulta de verificação

```
-- Quantos webhooks já trouxeram um referral de anúncio?
select
  count(*) filter (where msg ? 'referral')           as com_referral,
  count(*)                                           as total_mensagens,
  min(created_at)                                   as desde
from public.webhook_logs,
  lateral (select payload->'entry'->0->'changes'->0->'value'->'messages'->0) as m(msg)
where msg is not null;
```

☐ Passo 2: Decidir com base no resultado

<code>com_referral</code>	O que fazer
0	Pare. Reporte ao usuário: os anúncios não são Click-to-WhatsApp. O plano só faz sentido depois de mudar o destino do anúncio.
≥ 1	Siga para a Tarefa 1, e guarde uma amostra do payload (Passo 3)

☐ Passo 3: Guardar uma amostra real do formato

```
select payload->'entry'->0->'changes'->0->'value'->'messages'->0->'referral'
from public.webhook_logs
where payload->'entry'->0->'changes'->0->'value'->'messages'->0 ? 'referral'
limit 3;
```

Cole o resultado no PR. Os testes da Tarefa 2 devem usar **este** formato, não o que está documentado — a Meta muda campos sem avisar.

Tarefa 1 — Migration: tabelas de atribuição

Files:

- Criar: `supabase/migration_lead_attribution.sql`

Interfaces:

- Produz: tabelas `public.lead_attributions` e `public.meta_ad_map`, consumidas pelas

Tarefas 2, 3, 5, 6 e 7.

☐ **Passo 1: Escrever a migration**

```

-- Origem de cada conversa que chegou por anúncio. Tabela própria e não coluna em
-- `conversations` porque o referral chega uma vez só, na primeira mensagem, e a
-- conversa pode ser reaberta depois – queremos guardar as duas passagens.
create table if not exists public.lead_attributions (
  id                uuid default uuid_generate_v4() primary key,
  account_id        uuid not null references public.accounts(id) on delete cascade,
  conversation_id    uuid references public.conversations(id) on delete set null,
  patient_phone      text not null,
  -- ID do ANÚNCIO (não da campanha). A tradução vive em meta_ad_map.
  source_id          text not null,
  source_type        text not null default 'ad',
  -- Identificador único do clique. É o que torna o insert idempotente: a Meta
  -- reenvia o mesmo webhook quando não recebe 200 a tempo.
  ctwa_clid          text,
  headline           text,
  body               text,
  source_url          text,
  occurred_at        timestampz not null default now(),
  created_at         timestampz not null default now()
);

-- Idempotência do webhook. `ctwa_clid` pode vir nulo em anúncio antigo, por isso
-- o índice é parcial: sem clid caímos no segundo índice, por conversa.
create unique index if not exists idx_lead_attr_clid
  on public.lead_attributions(account_id, ctwa_clid)
  where ctwa_clid is not null;

create unique index if not exists idx_lead_attr_conversation
  on public.lead_attributions(conversation_id, source_id)
  where conversation_id is not null;

create index if not exists idx_lead_attr_lookup
  on public.lead_attributions(account_id, occurred_at desc);

alter table public.lead_attributions enable row level security;

drop policy if exists lead_attributions_select on public.lead_attributions;
create policy lead_attributions_select on public.lead_attributions
  for select using (account_id = any(public.my_account_ids()));

-- Tradução anúncio → campanha. O referral traz o ID do anúncio; ad_campaigns
-- guarda o ID da campanha. Sem esta tabela não dá para juntar os dois.
create table if not exists public.meta_ad_map (
  account_id        uuid not null references public.accounts(id) on delete cascade,
  ad_id             text not null,
  campaign_id        text not null,
  ad_name           text,
  adset_name         text,
  synced_at         timestampz not null default now(),
  primary key (account_id, ad_id)
);

alter table public.meta_ad_map enable row level security;

drop policy if exists meta_ad_map_select on public.meta_ad_map;

```

```
create policy meta_ad_map_select on public.meta_ad_map
for select using (account_id = any(public.my_account_ids()));
```

☐ Passo 2: Regenerar os tipos

```
npx supabase gen types typescript --linked > types/database.ts
```

Se o comando não estiver configurado, edite `types/database.ts` à mão seguindo o formato das tabelas vizinhas.

☐ Passo 3: Commit

```
git add supabase/migration_lead_attribution.sql types/database.ts
git commit -m "feat(trafego): tabelas de atribuicao de lead e mapa de anuncios"
```

Aviso ao executor: as políticas RLS só cobrem `SELECT`. O insert da Tarefa 2 roda no webhook, que usa o admin client — mesmo padrão de `workspace_ad_accounts`.

Tarefa 2 — Capturar o referral no webhook

Files:

- Criar: `lib/whatsapp/referral.ts`
- Criar: `tests/whatsapp/referral.test.ts`
- Modificar: `app/api/whatsapp/webhook/route.ts` (em volta da linha 84)

Interfaces:

- Produz: `parseReferral(message: unknown): ParsedReferral | null` e

```
recordAttribution(args: { accountId: string; patientPhone: string; referral: ParsedReferral }):
Promise<void>
```

- Consome: `lead_attributions` da Tarefa 1.

☐ Passo 1: Escrever o teste falhando

```
// tests/whatsapp/referral.test.ts
import { describe, it, expect } from 'vitest'
import { parseReferral } from '@lib/whatsapp/referral'

describe('parseReferral', () => {
  it('extraí a origem de uma mensagem vinda de anúncio', () => {
    const result = parseReferral({
      from: '5511999999999',
      referral: {
        source_id: '1202100000000',
        source_type: 'ad',
        source_url: 'https://fb.me/abc',
        headline: 'Consulta de rotina',
        body: 'Agende hoje',
        ctwa_clid: 'ARBx123',
      },
    },
  })

  expect(result).toEqual({
    sourceId: '1202100000000',
    sourceType: 'ad',
    sourceUrl: 'https://fb.me/abc',
    headline: 'Consulta de rotina',
    body: 'Agende hoje',
    ctwaClid: 'ARBx123',
  })
})

// A esmagadora maioria das mensagens é conversa normal. Se isso lançar,
// o webhook inteiro cai.
it('devolve nulo para mensagem comum, sem referral', () => {
  expect(parseReferral({ from: '5511999999999', text: { body: 'oi' } })).toBeNull()
})

it('devolve nulo quando o referral vem sem source_id', () => {
  expect(parseReferral({ referral: { source_type: 'ad' } })).toBeNull()
})

it('aceita referral sem ctwa_clid, que é o caso de anúncio antigo', () => {
  const result = parseReferral({ referral: { source_id: '1', source_type: 'ad' } })

  expect(result?.sourceId).toBe('1')
  expect(result?.ctwaClid).toBeNull()
})
})
```

☐ Passo 2: Rodar e confirmar que falha

```
npx vitest run tests/whatsapp/referral.test.ts
```

Esperado: `Failed to resolve import "@lib/whatsapp/referral"`.

☐ Passo 3: Implementar o mínimo

```
// lib/whatsapp/referral.ts

/** O que a Meta manda junto da PRIMEIRA mensagem de quem veio de um anúncio
 * Click-to-WhatsApp. Não vem nas mensagens seguintes da mesma conversa. */
export interface ParsedReferral {
  sourceId: string
  sourceType: string
  sourceUrl: string | null
  headline: string | null
  body: string | null
  ctwaClid: string | null
}

function str(value: unknown): string | null {
  return typeof value === 'string' && value.length > 0 ? value : null
}

export function parseReferral(message: unknown): ParsedReferral | null {
  const referral = (message as { referral?: Record<string, unknown> } | null)?.referral
  if (!referral) return null

  const sourceId = str(referral.source_id)
  // Sem o ID do anúncio a linha não serve para nada: é justamente ela que
  // fecha a ponte com a campanha.
  if (!sourceId) return null

  return {
    sourceId,
    sourceType: str(referral.source_type) ?? 'ad',
    sourceUrl: str(referral.source_url),
    headline: str(referral.headline),
    body: str(referral.body),
    ctwaClid: str(referral.ctwa_clid),
  }
}
```

☐ Passo 4: Rodar e confirmar que passa

```
npx vitest run tests/whatsapp/referral.test.ts
```

Esperado: 4 passando.

☐ Passo 5: Escrever o teste da gravação

```
// no mesmo arquivo
import { recordAttribution } from '@lib/whatsapp/referral'

describe('recordAttribution', () => {
  it('grava a origem ignorando conflito, porque a Meta reenvia o webhook', async () => {
    const upsert = vi.fn().mockResolvedValue({ error: null })
    const client = { from: vi.fn().mockReturnValue({ upsert }) }

    await recordAttribution(
      {
        accountId: 'acc1',
        patientPhone: '+5511999999999',
        referral: { sourceId: '1', sourceType: 'ad', sourceUrl: null,
          headline: null, body: null, ctwaClid: 'X' },
      },
      client as never
    )

    expect(client.from).toHaveBeenCalledWith('lead_attributions')
    expect(upsert).toHaveBeenCalledWith(
      expect.objectContaining({ account_id: 'acc1', source_id: '1', ctwa_clid: 'X' }),
      { onConflict: 'account_id,ctwa_clid', ignoreDuplicates: true }
    )
  })
})
```

☐ Passo 6: Rodar, confirmar falha, implementar `recordAttribution`

```
import { createAdminClient } from '@lib/supabase/admin'

export async function recordAttribution(
  args: { accountId: string; patientPhone: string; referral: ParsedReferral },
  client = createAdminClient()
): Promise<void> {
  const { error } = await client.from('lead_attributions').upsert(
    {
      account_id: args.accountId,
      patient_phone: args.patientPhone,
      source_id: args.referral.sourceId,
      source_type: args.referral.sourceType,
      source_url: args.referral.sourceUrl,
      headline: args.referral.headline,
      body: args.referral.body,
      ctwa_clid: args.referral.ctwaClid,
    },
    { onConflict: 'account_id,ctwa_clid', ignoreDuplicates: true }
  )
  // Não relançamos: perder uma atribuição é ruim, perder a resposta ao paciente
  // é pior. O webhook precisa devolver 200 em menos de 20s de qualquer jeito.
  if (error) console.error('[whatsapp] recordAttribution falhou', error)
}
```

Confirme o nome real do helper do admin client antes de escrever o import — procure por `createAdminClient` em `lib/supabase/`.

☐ Passo 7: Ligar no webhook

Em `app/api/whatsapp/webhook/route.ts`, logo depois de `const message = value.messages[0]` e dentro do `after(...)` que já existe (a Meta exige resposta em menos de 20s):

```
const referral = parseReferral(message)
if (referral) {
  await recordAttribution({ accountId, patientPhone: from, referral })
}
```

☐ Passo 8: Rodar a suíte inteira e commitar

```
npm test
git add lib/whatsapp/referral.ts tests/whatsapp/referral.test.ts app/api/whatsapp/webhook/route.ts
git commit -m "feat(trafego): captura a origem do lead que chega por anuncio"
```

Tarefa 3 — Traduzir anúncio → campanha no sync

Files:

- Modificar: `lib/meta/ads-sync.ts`
- Modificar: `tests/meta/ads-sync.test.ts`

Interfaces:

- Consome: `fetchAllInsights` e `graphFetch`, já existentes em `lib/meta/ads-sync.ts`.
- Produz: linhas em `meta_ad_map`, consumidas pela Tarefa 5.

O `referral.source_id` é o ID do anúncio. O `ad_campaigns.external_campaign_id` guarda o ID da campanha. Sem a tradução, as duas metades nunca se encontram.

☐ Passo 1: Escrever o teste falhando

```
it('grava o mapa anuncio -> campanha junto do sync', async () => {
  // 1ª chamada: insights por campanha (o que já existia).
  // 2ª chamada: insights por anúncio, só para o mapa.
  fetchMock
    .mockResolvedValueOnce(page([row('c1', '2026-09-17')], null))
    .mockResolvedValueOnce(
      page([
        { ad_id: 'a1', ad_name: 'Criativo A', campaign_id: 'c1' }
      ], null)
    )

  await syncAdsForAccount('acc1', { days: 7 })

  expect(upsertedInto('meta_ad_map')).toContainEqual(
    expect.objectContaining({ account_id: 'acc1', ad_id: 'a1', campaign_id: 'c1' })
  )
})
```

Ajuste `upsertedInto` ao helper de mock que o arquivo já usa para `ad_campaigns`.

☐ Passo 2: Rodar e confirmar que falha

```
npx vitest run tests/meta/ads-sync.test.ts
```

☐ Passo 3: Implementar

```
interface AdRow {
  ad_id?: string
  ad_name?: string
  adset_name?: string
  campaign_id?: string
}

/** Segunda passada no mesmo endpoint, agora em `level: 'ad'`. Só precisamos dos
 * identificadores, então `time_increment` fica de fora: uma linha por anúncio
 * no período inteiro, não uma por dia. */
async function fetchAdMap(
  adAccountId: string, token: string, timeRange: string
): Promise<AdRow[]> {
  const data = await graphFetch<{ data?: AdRow[] }>(`/${adAccountId}/insights`, {
    token,
    params: {
      level: 'ad',
      time_range: timeRange,
      fields: 'ad_id,ad_name,adset_name,campaign_id',
      limit: '500',
    },
  })
  return data.data ?? []
}
```

E, dentro do laço de mapeamento que já existe, depois do upsert de `ad_campaigns`:

```
const ads = await fetchAdMap(mapping.ad_account_id, token, timeRange)
const adPayload = ads
  .filter((ad) => ad.ad_id && ad.campaign_id)
  .map((ad) => ({
    account_id: accountId,
    ad_id: ad.ad_id!,
    campaign_id: ad.campaign_id!,
    ad_name: ad.ad_name ?? null,
    adset_name: ad.adset_name ?? null,
    synced_at: new Date().toISOString(),
  }))

if (adPayload.length > 0) {
  await admin.from('meta_ad_map').upsert(adPayload, { onConflict: 'account_id,ad_id' })
}
```

☐ Passo 4: Rodar e confirmar que passa

```
npx vitest run tests/meta/
```

Esperado: os 73 testes anteriores + o novo.

☐ Passo 5: Commit

```
git add lib/meta/ads-sync.ts tests/meta/ads-sync.test.ts
git commit -m "feat(trafego): sincroniza o mapa de anuncio para campanha"
```

Tarefa 4 — Backfill do histórico

Files:

- Criar: `scripts/backfill-referrals.sql`

Recupera as atribuições que passaram pelo webhook antes da Tarefa 2 existir. Sem isso, a tela fica vazia até o próximo anúncio rodar.

☐ Passo 1: Escrever o script

```
-- Idempotente: pode rodar quantas vezes quiser (o ON CONFLICT cobre).
insert into public.lead_attributions
(account_id, patient_phone, source_id, source_type, source_url,
 headline, body, ctwa_clid, occurred_at)
select
bc.account_id,
'+' || (m.msg ->> 'from'),
r ->> 'source_id',
coalesce(r ->> 'source_type', 'ad'),
r ->> 'source_url',
r ->> 'headline',
r ->> 'body',
r ->> 'ctwa_clid',
to_timestamp((m.msg ->> 'timestamp')::bigint)
from public.webhook_logs wl
cross join lateral (
select wl.payload -> 'entry' -> 0 -> 'changes' -> 0 -> 'value' -> 'messages' -> 0
) as m(msg)
cross join lateral (select m.msg -> 'referral') as ref(r)
join public.bot_config bc
on bc.phone_number_id =
wl.payload -> 'entry' -> 0 -> 'changes' -> 0 -> 'value' -> 'metadata' ->> 'phone_number_id'
where r is not null
and r ->> 'source_id' is not null
on conflict do nothing;
```

☐ Passo 2: Conferir em seco antes de escrever

Troque o `insert into ... select` por `select` puro e confira: as contas batem? Os telefones saem em E.164? Se `bot_config.phone_number_id` não for o nome real da coluna, corrija — confirme com `\d public.bot_config`.

☐ Passo 3: Rodar e commitar o script

```
git add scripts/backfill-referrals.sql
git commit -m "chore(trafego): script de backfill das atribuicoes historicas"
```

Tarefa 5 — Montar a cadeia de atribuição

Files:

- Criar: `lib/trafego/attribution.ts`
- Criar: `tests/trafego/attribution.test.ts`

Interfaces:

- Produz o tipo consumido pelas Tarefas 6, 7 e 8:

```
export interface AttributedLead {
  campaignId: string | null    // null = anúncio sem mapa ainda
  campaignName: string | null
  channel: string
  patientPhone: string
  leadAt: string               // ISO - data do lead, ancora o funil
  appointments: { status: string; scheduledAt: string }[]
  paidRevenue: number         // soma de revenue_entries pagas na janela
  forecastRevenue: number
}
```

- ☐ Passo 1: Escrever o teste falhando

```

import { describe, it, expect } from 'vitest'
import { buildLeads } from '@/lib/trafego/attribution'

const ATTR = {
  patient_phone: '+5511999999999', source_id: 'a1',
  occurred_at: '2026-09-01T10:00:00Z',
}

describe('buildLeads', () => {
  it('liga o lead à campanha pelo mapa de anúncios', () => {
    const leads = buildLeads({
      attributions: [ATTR],
      adMap: [{ ad_id: 'a1', campaign_id: 'c1' }],
      campaigns: [{ external_campaign_id: 'c1', campaign_name: 'Implantes', channel: 'facebook' }],
      appointments: [],
      revenue: [],
    })

    expect(leads[0]).toMatchObject({ campaignId: 'c1', campaignName: 'Implantes' })
  })

  // Anúncio criado depois do último sync ainda não está no mapa. O lead existe
  // e precisa aparecer – some da tabela por campanha, não do total de leads.
  it('mantém o lead mesmo sem mapa do anúncio', () => {
    const leads = buildLeads({
      attributions: [ATTR], adMap: [], campaigns: [], appointments: [], revenue: [],
    })

    expect(leads).toHaveLength(1)
    expect(leads[0].campaignId).toBeNull()
  })

  it('só conta consulta dentro da janela de 90 dias após o lead', () => {
    const leads = buildLeads({
      attributions: [ATTR],
      adMap: [{ ad_id: 'a1', campaign_id: 'c1' }],
      campaigns: [],
      appointments: [
        { patient_phone: '+5511999999999', status: 'realizado', scheduled_at: '2026-09-10T10:00:00Z' },
        { patient_phone: '+5511999999999', status: 'realizado', scheduled_at: '2027-06-01T10:00:00Z' },
      ],
      revenue: [],
    })

    expect(leads[0].appointments).toHaveLength(1)
  })

  it('ignora consulta anterior ao lead – não foi o anúncio que a trouxe', () => {
    const leads = buildLeads({
      attributions: [ATTR],
      adMap: [], campaigns: [], revenue: [],
      appointments: [
        { patient_phone: '+5511999999999', status: 'realizado', scheduled_at: '2026-08-01T10:00:00Z' },
      ],
    })
  })
})

```

```
    expect(Leads[0].appointments).toHaveLength(0)
  })
})
```

☐ Passo 2: Rodar e confirmar que falha

```
npx vitest run tests/trafego/attribution.test.ts
```

☐ Passo 3: Implementar

Puro, sem I/O — toda a busca no Supabase fica na página (Tarefa 8), como em `lib/trafego/aggregate.ts`. Regras a codificar:

1. Indexe `adMap` por `ad_id` e `campaigns` por `external_campaign_id`.
2. Para cada atribuição, resolva `campaignId` (pode ser `null`).
3. Case consultas por `patient_phone`, aceitando só `scheduled_at` entre `leadAt` e `leadAt + 90 dias` (`ATTRIBUTION_WINDOW_DAYS = 90`, exportada).
1. Some `revenue_entries` dos `appointment_id` casados, separando `payment_status === 'paid'` de `'pending'`.
1. Telefone duplicado com duas atribuições: fique com a **mais recente** (last touch).

☐ Passo 4: Rodar, confirmar verde, commitar

```
npx vitest run tests/trafego/
git add lib/trafego/attribution.ts tests/trafego/attribution.test.ts
git commit -m "feat(trafego): monta a cadeia de anuncio ate receita"
```

Tarefa 6 — Funil de conversão

Files:

- Criar: `lib/trafego/funnel.ts`, `tests/trafego/funnel.test.ts`
- Criar: `components/trafego/FunnelCard.tsx`

Interfaces:

- Consome: `AttributedLead[]` da Tarefa 5.
- Produz: `funnelSteps(leads: AttributedLead[]): FunnelStep[]` com `{ label: string; value: number; pct: number }`.

As quatro etapas, com a regra exata de cada uma:

Etapa	Regra
Leads	toda <code>AttributedLead</code> no período
Agendamentos	lead com ≥ 1 consulta em <code>agendado</code> , <code>confirmado</code> ou <code>realizado</code>
Consultas realizadas	lead com ≥ 1 consulta <code>realizado</code>
Pacientes recorrentes	lead com ≥ 2 consultas <code>realizado</code>

☐ Passo 1: Escrever o teste falhando

```
import { describe, it, expect } from 'vitest'
import { funnelSteps } from '@lib/trafego/funnel'

const lead = (statuses: string[]) => ({
  campaignId: 'c1', campaignName: 'x', channel: 'facebook',
  patientPhone: '+551199', leadAt: '2026-09-01T10:00:00Z',
  appointments: statuses.map((status) => ({ status, scheduledAt: '2026-09-05T10:00:00Z' })),
  paidRevenue: 0, forecastRevenue: 0,
})

describe('funnelSteps', () => {
  it('conta cada etapa uma vez por lead, não por consulta', () => {
    // Um lead com três consultas realizadas não são três agendamentos.
    const steps = funnelSteps([lead(['realizado', 'realizado', 'realizado'])])

    expect(steps.map((s) => s.value)).toEqual([1, 1, 1, 1])
  })

  it('cancelada e no_show não contam como agendamento', () => {
    const steps = funnelSteps([lead(['cancelado']), lead(['no_show'])])

    expect(steps[1].value).toBe(0)
  })

  it('recorrente exige duas consultas realizadas', () => {
    const steps = funnelSteps([lead(['realizado'])])

    expect(steps[3].value).toBe(0)
  })

  it('o percentual é sempre sobre o total de leads, não sobre a etapa anterior', () => {
    const steps = funnelSteps([lead(['realizado']), lead([]), lead([]), lead([])])

    expect(steps[2].pct).toBe(25)
  })

  it('sem lead nenhum devolve quatro etapas zeradas, não uma lista vazia', () => {
    expect(funnelSteps([])).toHaveLength(4)
    expect(funnelSteps([])[0].pct).toBe(0)
  })
})
```

☐ Passo 2: Rodar e confirmar que falha

- ☐ Passo 3: Implementar `funnelSteps`
- ☐ Passo 4: Rodar e confirmar que passa
- ☐ Passo 5: Escrever o `FunnelCard`

Barras horizontais, mesmo desenho do design: rótulo e valor na linha de cima, trilho `var(--navy-06)` com preenchimento `var(--cyan)`. Cartão no padrão já usado em `components/trafego/LeadsChart.tsx`.

Aviso obrigatório na UI. Um funil ancorado na data do lead sempre parece terrível em janela curta: o lead de ontem ainda não virou consulta. Quando `days === 7`, mostre abaixo do card:

"Leads dos últimos 7 dias ainda não tiveram tempo de virar consulta. Use 30 ou 90 dias para ler a conversão."

- ☐ Passo 6: Commit

```
git add lib/trafego/funnel.ts tests/trafego/funnel.test.ts components/trafego/FunnelCard.tsx
git commit -m "feat(trafego): funil de conversao do lead ate paciente recorrente"
```

Tarefa 7 — ROI por campanha

Files:

- Criar: `lib/trafego/roi.ts`, `tests/trafego/roi.test.ts`

Interfaces:

- Consome: `AttributedLead[]` (Tarefa 5) e `CampaignTotal[]` (`lib/trafego/aggregate.ts`).
- Produz: `roiByCampaign(leads, totals): Map<string, RoiResult>` com

```
{ revenue: number; roi: number | null }.
```

- ☐ Passo 1: Escrever o teste falhando

```
import { describe, it, expect } from 'vitest'
import { roiByCampaign } from '@lib/trafego/roi'

describe('roiByCampaign', () => {
  it('divide a receita paga pelo investimento da campanha', () => {
    const result = roiByCampaign(
      [{ campaignId: 'c1', paidRevenue: 3000, forecastRevenue: 0 } as never],
      [{ externalId: 'c1', spend: 1000 }]
    )

    expect(result.get('c1')).toMatchObject({ revenue: 3000, roi: 3 })
  })

  // Investimento zero com receita > 0 daria Infinity e imprimiria "∞x" na tela.
  it('com investimento zero o ROI é nulo, não infinito', () => {
    const result = roiByCampaign(
      [{ campaignId: 'c1', paidRevenue: 500, forecastRevenue: 0 } as never],
      [{ externalId: 'c1', spend: 0 }]
    )

    expect(result.get('c1')?.roi).toBeNull()
  })

  it('campanha que gastou e não gerou receita tem ROI zero, não nulo', () => {
    const result = roiByCampaign([], [{ externalId: 'c1', spend: 800 }])

    expect(result.get('c1')?.roi).toBe(0)
  })

  it('não conta receita prevista no ROI principal', () => {
    const result = roiByCampaign(
      [{ campaignId: 'c1', paidRevenue: 0, forecastRevenue: 9000 } as never],
      [{ externalId: 'c1', spend: 1000 }]
    )

    expect(result.get('c1')?.roi).toBe(0)
  })
})
```

☐ Passo 2: Rodar, confirmar falha, implementar, confirmar verde

☐ Passo 3: Commit

```
git add lib/trafego/roi.ts tests/trafego/roi.test.ts
git commit -m "feat(trafego): roi por campanha a partir da receita paga"
```

Limite honesto, que precisa aparecer na tela: consulta por convênio não gera `revenue_entry` (está escrito no schema, em `appointments.health_plan`). Numa clínica que atende convênio o ROI sai subestimado. Coloque uma nota de rodapé no card: *"Considera apenas receita particular registrada e paga."* Sem essa linha, o número mente.

Tarefa 8 — Ligar tudo na tela

Files:

- Modificar: `app/(dashboard)/trafego/page.tsx`
- Modificar: `components/trafego/CampaignsClient.tsx`
- Criar: `components/trafego/AttributionList.tsx`

☐ Passo 1: Buscar a cadeia na página

Em `page.tsx`, junto da busca de `ad_campaigns` que já existe, adicione as buscas de `lead_attributions`, `meta_ad_map`, `appointments` e `revenue_entries` — todas já recortadas pelos mesmos 90 dias da constante `MAX_WINDOW_DAYS`. Passe o resultado de `buildLeads` como prop serializável para o cliente.

☐ Passo 2: Quarto KPI

Volta o card de ROI, agora com barra legítima (ao contrário dos três primeiros, cuja barra foi removida na Fase 1 por ser decorativa): largura `min(100, roi / 5 * 100)`, cor `var(--success)`. Rótulo: "ROI (retorno sobre investimento)". Valor: `2,4x`, ou `–` quando nulo.

☐ Passo 3: Grid do gráfico com o funil

Trocar o `LeadsChart` de largura cheia para o grid `2fr 1fr` do design, com o `FunnelCard` na coluna da direita. Em telas estreitas, empilhar.

☐ Passo 4: Coluna ROI na tabela

Adicionar a coluna à direita de CPL, alinhada à direita, lendo o `Map` da Tarefa 7.

☐ Passo 5: Lista de atribuição

O design chama de "Atribuição por origem" e mostra `meta / cpc` — vocabulário de UTM. **Não existe UTM aqui:** o lead chega pelo WhatsApp, não por uma landing page com `?utm_source=`. Renomeie o card para "Atribuição por campanha" e mostre `Canal · Nome da campanha` com a contagem de leads e o percentual do total. Inventar `meta / cpc` seria imitar a aparência de um dado que o sistema não tem.

Se um dia existir landing page própria, aí sim cabe capturar `utm_source / utm_medium / utm_campaign` de verdade — e esse card já estará no lugar certo para recebê-los.

☐ Passo 6: Verificar e commitar

```
npm test && npx tsc --noEmit && npm run lint && npm run build
git add app/(dashboard)/trafego/page.tsx components/trafego/
git commit -m "feat(trafego): roi, funil e atribuicao por campanha na tela"
```

Os 3 erros de lint em `Invite email HTML design/support.js` e `app/(admin)/admin/costs/page.tsx` são anteriores a este trabalho. Não os conserte aqui.

Riscos

Risco	Sinal de que aconteceu	O que fazer
Anúncios não são Click-to-WhatsApp	Tarefa 0 volta <code>com_referral = 0</code>	Pare. O problema é de mídia, não de código.
Paciente escreve de outro número	Lead sem consulta, mas a consulta existe no sistema	Aceitar a perda; casar por nome é pior que não casar
<code>source_id</code> sem mapa	Leads com <code>campaignId: null</code>	Rodar o sync; se persistir, o anúncio foi excluído na Meta
Clínica majoritariamente de convênio	ROI ridículamente baixo em todas as campanhas	A nota de rodapé da Tarefa 7 é o que impede a leitura errada
<code>webhook_logs</code> crescer demais com o backfill	Consulta da Tarefa 4 lenta	Rodar em lotes por <code>created_at</code>

Ordem de execução

Tarefas **0** → **1** → **2** → **3** → **4** são uma corrente: cada uma depende da anterior. A Tarefa 4 pode ser adiada sem bloquear nada (só adia a chegada dos dados históricos).

Tarefas **6** e **7** são independentes entre si e ambas dependem da 5. A Tarefa 8 é a última.

Depois da Tarefa 2 ir para produção, **espere alguns dias de dados reais** antes de executar da 5 em diante — é muito mais fácil acertar a agregação olhando para dez leads reais do que para um fixture inventado.